

Revisão de C

Laboratórios de Informática III

Guião #1

Departamento de Informática
Universidade do Minho

2025/26

Conteúdo

1	Conceitos	2
1.1	Função <code>main</code>	2
1.2	Inclusão de ficheiros	2
1.3	Arrays	3
1.4	Structs	3
1.5	Escopo e tempo de vida de variáveis	4
1.6	Apontadores	5
1.7	Gestão de memória	8
1.8	<code>#define</code> e <code>typedef</code>	9
1.9	Estruturas de dados abstratas	11
1.10	Ficheiros <code>.h</code> vs <code>.c</code>	12
1.11	Compilação e execução	14
2	Exemplo prático	16
3	Exercícios	19
4	Conceitos adicionais	20

1 Conceitos

1.1 Função `main`

A função `main` é o ponto de entrada de qualquer programa em C, tendo a seguinte assinatura¹:

```
1 int main(int argc, char** argv);
```

É uma função que retorna um inteiro (normalmente 0, indicando que o programa executou com sucesso) e recebe dois parâmetros:

- `argc` – número de argumentos passados ao programa;
- `argv` – array de argumentos em formato textual.

O primeiro elemento de `argv` é reservado para o nome do executável e o último é sempre o endereço nulo (`NULL`), respetivamente, `argv[0]` e `argv[argc]`.

Considere um programa `repeat`, muito simples, que recebe como argumentos uma string e o número de vezes que deverá ser impressa na consola:

```
1 #include <stdlib.h> // para usar o "atoi"
2 #include <stdio.h> // para usar o "printf"
3
4 int main(int argc, char** argv) {
5     printf("%s recebeu %d argumentos.\n", argv[0], argc);
6     int repeats = atoi(argv[2]); // atoi converte uma string para um inteiro
7     while (repeats > 0) {
8         printf("%s\n", argv[1]);
9         repeats--;
10    }
11    return 0;
12 }
```

Usando o `gcc`, podemos compilar o programa da seguinte forma:

```
$ gcc repeat.c -o repeat
```

Executando `./repeat "Hello, world!" 4` obtemos o seguinte output:

```
./repeat recebeu 3 argumentos.
Hello, world!
Hello, world!
Hello, world!
Hello, world!
```

1.2 Inclusão de ficheiros

Antes de usarmos funções ou tipos de dados externos ao nosso código-fonte devemos sempre – e em muitos casos *temos* mesmo – que incluir as respetivas declarações. Por exemplo, se queremos usar a função `printf`, devemos incluir a sua declaração, a qual, por convenção da biblioteca de C, encontra-se no ficheiro `stdio.h`. A diretiva `#include` permite-nos importar código de ficheiros externos para que o possamos usar no nosso ficheiro. Existem duas variantes para importação de ficheiros. A primeira variante vai procurar o ficheiro `file.h` num conjunto de caminhos pré-definidos:

```
1 #include <file.h>
```

A segunda variante irá primeiramente procurar o ficheiro `file.h` na mesma pasta em que o nosso ficheiro se encontra. Caso não o encontre, irá procurar o ficheiro da mesma maneira que a variante anterior:

```
1 #include "file.h"
```

¹Em alternativa mas de forma equivalente, o argumento `argv` também pode ser declarado como `char* argv[]`.

1.3 Arrays

Um array é uma estrutura de dados que nos permite agrupar dados do mesmo tipo em posições contíguas de memória. Acessos a posições de memória fora dos limites do array podem levar a comportamentos indesejados, como a alteração dos valores de outras variáveis, ou causar a terminação abrupta da execução do programa. Por isso, os limites de tamanho do array devem ser respeitados. O seguinte excerto apresenta exemplos de declarações de arrays:

```
1 int int_array1[10]; // array de inteiros com espaço para 10 elementos
2 int int_array2[] = {1,2,3,4,5,6,7,8,9,10}; // igual ao exemplo anterior, mas as 10 posições são
   ↳ preenchidas no momento de declaração da variável; declarar o tamanho do array é opcional, pois o
   ↳ compilador consegue inferi-lo
3 int int_array3[10] = {1,2,3}; // as restantes posições são inicializadas com o valor 0
```

Arrays de caracteres podem ser instanciados também a partir de *string literals*. O conteúdo da string imutável é copiado para as posições do array, mas é importante notar que isto só é possível na inicialização da variável. Após a inicialização, a string contida no array poderá ser copiada – como qualquer string – usando a função `strcpy`. O compilador consegue inferir o tamanho de um array a partir de um string literal:

```
1 char char_array1[10];
2 char char_array2[] = {'a','b','c','d','e','f','g','h','i','\0'};
3 char char_array3[] = "abcdefghi"; // forma alternativa; copia o string literal para as posições do
   ↳ array
```

Valores de um array podem ser acedidos através de índices que começam no valor 0 e se estendem até [tamanho do array - 1]:

```
1 char char_array[10];
2 char_array[0] = 1; // escrita no primeiro elemento do array
3 char_array[9] = 5; // escrita no último elemento do array
```

É importante perceber que uma variável de array não é um apontador. Para mais detalhes sobre a distinção entre os dois, ver secção 1.6.

1.4 Structs

As structs em C permitem agrupar um conjunto de variáveis logicamente. Por exemplo, se quisermos representar os atributos de um produto de uma loja, podemos usar a seguinte struct:

```
1 struct item {
2     char name[30];
3     int stock;
4     double basePrice;
5     double discount; // [0, 1[
6 };
```

Para criarmos um produto, basta usar `struct item i;`. Para aceder às variáveis de uma struct, usamos o operador `.`:

```
1 struct item i;
2 strcpy(i.name, "item 1"); // <string.h> precisamos de copiar a string para a struct
3 i.basePrice = 9.99;
4 i.stock = 14;
5 i.discount = 0;
```

1.5 Escopo e tempo de vida de variáveis

O escopo de uma variável define as **zonas do código em que esta pode ser manipulada**. O tempo de vida de uma variável define o **período durante o qual a variável se mantém alocada em memória**.

Quando uma variável é definida dentro de uma função, sem qualquer modificador, esta só pode ser manipulada dentro dessa função e o seu tempo de vida será o tempo de execução da função. A isto chamamos uma variável local:

```

1 void func2() {
2     // não podemos manipular num1, porque a func2 está fora do seu scope, mas ainda está alocado em
   ↪ memória no momento de execução desta função (tempo de vida estende-se a func2)
3     num1 = 1; // erro de compilação
4 }
5
6 void func1() {
7     int num1;
8     // podemos manipular num1
9     num1 = 10;
10    func2();
11 }

```

Duas variáveis locais com o mesmo nome podem existir no mesmo programa, desde que não se sobreponham no seu escopo, ou seja, desde que sejam declaradas em funções diferentes:

```

1 // cada uma das variáveis num1 tem o scope local à sua função, e não é afetada pelas operações da
   ↪ outra função
2
3 void func1() {
4     int num1;
5     num1 = 10;
6 }
7
8 void func2() {
9     int num1;
10    num1 = 10+3;
11 }

```

Uma variável pode também ser definida fora de uma função. Neste caso, o escopo da variável estende-se a todas as funções do programa e tem um tempo de vida que se estende até à conclusão da execução do programa, atribuindo-se-lhe o nome de variável global:

```

1 int a;
2
3 void func1() {
4     a = a + 2;
5 }
6
7 void func2() {
8     a++;
9 }
10
11 int main(int argc, char** argv) {
12     a = 0;
13     func1(); // a passa a ter valor 2
14     func2(); // a passa a ter valor 3
15 }

```

Quando duas variáveis, com o mesmo nome, se sobrepõem no mesmo escopo de tal forma que uma variável é local, e a outra é global, a variável local irá sobrepor-se à global:

```

1  int a;
2
3  void func1() {
4      int a = 0;
5      a = a + 2; // a tem valor 2
6  }
7
8  void func2() {
9      a++; // a tem valor 4
10 }
11
12 int main(int argc, char** argv) {
13     a = 3;
14     func1(); // variável global 'a' não é alterada nesta função
15     func2(); // variável global 'a' passa a ter valor 4
16 }

```

1.6 Apontadores

As variáveis em C podem armazenar diferentes tipos de dados, como `int`, `float`, `char`, `long long`, entre outros. Contudo, também podem guardar um endereço de memória (e.g., o local onde uma variável está guardada), sendo designados por “apontadores”. Os apontadores são representados por `tipo*`, e.g., `int*`. Podemos usar o símbolo `*` num apontador para aceder ao seu valor (i.e., desreferenciar o apontador) e o símbolo `&` para obter o endereço de uma variável:

```

1  int x = 10; // x é um inteiro de valor 10
2  int* y = &x; // y é um apontador para um inteiro
3  printf("Endereço de x = %p\n", y); // endereço de x = 0x7fffea9d58ac
4  printf("y = %d\n", *y); // *y = 10
5  x *= 2;
6  printf("y = %d\n", *y); // *y = 20

```

Os apontadores tornam-se especialmente úteis na leitura e modificação de estruturas de dados mais complexas, sobretudo quando queremos manipular as mesmas estruturas de dados a partir de funções diferentes, sem ter que efetuar várias cópias. Por exemplo:

```

1  struct house {
2      char street[150];
3      int area;
4      char owner[50];
5  }
6
7  void fill_house_details(struct house* my_house) {
8      strcpy((*my_house).street, "foo");
9      (*my_house).area = 50;
10     strcpy((*my_house).owner, "bar");
11 }
12
13 int main(int argc, char** argv) {
14     struct house my_house;
15     fill_house_details(&my_house);
16 }

```

A estrutura `struct house` é instanciada apenas uma vez. Passando o endereço da estrutura em vez da própria estrutura evita-se que sejam feitas múltiplas cópias. É necessário, no entanto, ter cuidado com a partilha de estruturas de dados através de apontadores, uma vez que as funções a quem são passados os apontadores podem modificar autonomamente o conteúdo das estruturas.

Aritmética de apontadores

Tal como com valores numéricos, também é possível aplicar operações aritméticas a apontadores. No entanto, estas operações não são idênticas a operações numéricas (e.g., somar 1 a um apontador não incrementa necessariamente o valor do apontador em 1 unidade). Por exemplo, considere o seguinte array:

```
1 char l[] = {'H', 'e', 'l', 'l', 'o', '\0'}; // \0 é o char NULL
2 char* l_ = l; // l_ contém o endereço para o primeiro elemento do array l
```

Se quisermos aceder ao primeiro elemento de `l`, basta fazer `*l`; para aceder ao segundo, fazemos `*(l+1)`; para aceder ao n -ésimo, fazemos `*(l+n-1)`. Sendo assim, podemos definir a função `strlen`, que calcula o tamanho de uma string, da seguinte forma:

```
1 void strlen_(char* s) {
2     int length = 0;
3     for (; *s; s++, length++);
4     return length;
5 }
```

Neste caso, somar 1 ao apontador incrementa o endereço num byte, dado que o tamanho de `char` é 1. Contudo, nem todos os valores têm tamanho 1. Por exemplo, considere o seguinte array de `short`²:

```
1 short s[] = {123, 456, 789};
```

Neste caso, a operação `s += 1` irá incrementar o endereço guardado em `s` 2 unidades (dado que o tamanho de um `short` é 2 bytes), como podemos comprovar no seguinte exemplo e no esquema da Figura 1:

```
1 printf("l:%p; l+1:%p\n", l, l+1); // l:0x7ffff1b1b5f3; l+1:0x7ffff1b1b5f4
2 printf("s:%p; s+1:%p\n", s, s+1); // s:0x7ffff1b1b5ec; s+1:0x7ffff1b1b5ee
```

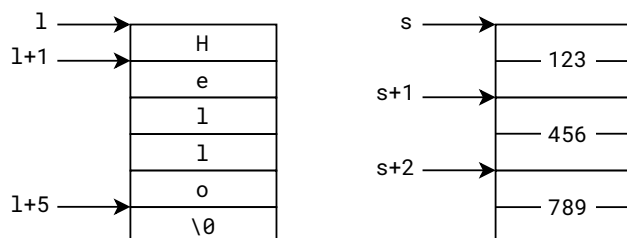


Figura 1: Exemplo da estrutura de dois arrays, `char* l` e `short* s`.

Por último, é relevante salientar o apontador especial `void*`, que pode apontar para qualquer tipo de dados. Este apontador é usado sobretudo na implementação de estruturas de dados genéricas, como iremos ver mais à frente na Secção 1.9. Dado que pode referir-se a qualquer valor, não devemos somar ou subtrair diretamente ao apontador sem antes o converter para o seu tipo concreto (cast).

Apontadores de structs

Se tivermos um apontador para uma struct, podemos aceder ao seu conteúdo da forma anteriormente abordada. Ou seja, desreferenciando o apontador através de `*` e depois usando o operador `.`. Contudo, podemos simplificar o acesso a elementos de uma struct através do seu apontador, usando `->`. No seguinte exemplo, as duas funções são equivalentes:

² `short` representa um valor numérico inteiro com mínimo -32768 e máximo 32767.

```

1 void printItemDereference(struct item* i) {
2     printf("{\n\tNome: %s,\n\tPreço base: %.02f,\n\tStock: %d,\n\tDesconto: %d%%\n}\n",
3         (*i).name, (*i).basePrice, (*i).stock, (int)((*i).discount * 100));
4 }
5
6 void printItemArrow(struct item* i) {
7     printf("{\n\tNome: %s,\n\tPreço base: %.02f,\n\tStock: %d,\n\tDesconto: %d%%\n}\n",
8         i->name, i->basePrice, i->stock, (int)(i->discount * 100));
9 }

```

Arrays vs apontadores

É importante notar que uma variável do tipo array, ainda que semelhante, não é idêntica a um apontador para o início desse array. Uma variável do tipo array aloca espaço para todos os elementos de um array. Um apontador apenas aloca espaço para um endereço. Uma variável do tipo array, quando lhe é aplicada a função `sizeof`, irá retornar o tamanho do array inteiro, enquanto que um apontador retornaria o tamanho de um endereço. Para além disso, variáveis do tipo array são imutáveis. Os seus elementos podem ser modificados, mas não a própria variável:

```

1 int arr1[10];
2 int arr2[12] = {1,2,3,4,5};
3 int* arr_pointer;
4
5 arr1 = arr2; // operação não permitida! O valor de arr1 é imutável!
6 arr_pointer = arr1; // arr1 é implicitamente convertido para o endereço do primeiro elemento do
   ↳ array; operação válida.
7
8 sizeof(arr1); // devolve 10*sizeof(int)
9 sizeof(arr_pointer) // devolve o tamanho de um endereço de memória
10
11 //Um caso interessante
12 char* string = "Hello world"; // string literals são alocados em memória apenas de leitura; uma vez
   ↳ que a variável string não se trata de um array, vai apenas apontar para o local onde o string
   ↳ literal está alocado, não sendo possível, por esse motivo, alterar o valor da string

```

Quando um array é passado como argumento para outra função, é implicitamente convertido no endereço do seu primeiro elemento. Assim, o valor retornado por `sizeof` quando aplicado ao argumento será o de um endereço, e pode ser-lhe atribuído um valor diferente. Esta conversão implícita acontece também quando imprimimos o valor da variável do tipo array:

```

1 void called_function(int arr1[], int* arr2) {
2     // as duas variáveis são apontadores
3     sizeof(arr1); // devolve o tamanho de um endereço (warning: 'sizeof' on array function parameter
   ↳ 'arr1' will return size of 'int*')
4     sizeof(arr2); // devolve o tamanho de um endereço
5     int arr3[10];
6     arr1 = arr3; // operação permitida, mas não correta, dado que a referência a arr3 tornar-se-á
   ↳ inválida quando a função terminar
7 }
8
9 int main(int argc, char** argv) {
10     int arr1[10];
11     int arr2[10];
12     called_function(arr1, arr2);
13     return 0;
14 }

```

1.7 Gestão de memória

A Figura 2 mostra uma visão geral da estrutura de memória de um programa C. Existem duas secções de memória de tamanho variável, a Stack e a Heap. O facto destas secções serem variáveis significa que podem crescer ou diminuir ao longo da execução do programa, tal como mostra a Figura 2:

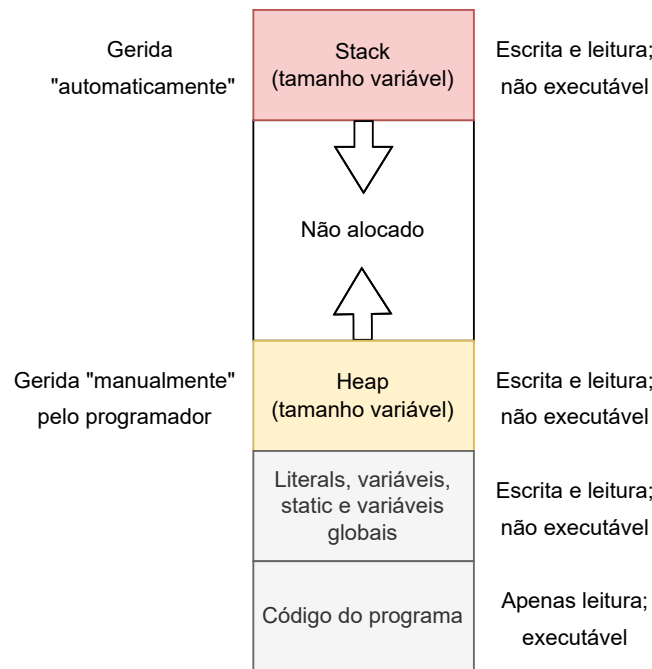


Figura 2: Estrutura de memória de um programa C

A Stack pode-se considerar como sendo "gerida automaticamente" pelo programa. Quando uma função é chamada, a Stack cresce. É alocado espaço para as variáveis locais da função, para os argumentos passados à função, e para outros elementos necessários ao normal funcionamento do programa. Quando uma função termina, a Stack encolhe. O espaço alocado para as variáveis locais e argumentos é libertado, mantendo-se apenas o valor retornado pela função. Tudo isto é feito sem qualquer intervenção explícita do programador.

Já a Heap é controlada pelos comandos `malloc`, `calloc`, `realloc` e `free`. A Heap só cresce ou encolhe quando é explicitamente manipulada pelo programador, através das funções anteriormente descritas. A terminação de funções não afeta a alocação de variáveis na Heap.

Vejamos os exemplos abaixo:

```

1 // na Stack é alocado espaço para as variáveis 'result', 'constant', e para os argumentos 'a' e 'b';
  ↳ os argumentos 'a' e 'b' são na verdade cópias dos valores que lhe foram passadas pela main
2 int const_sum_values(int a, int b) {
3     int constant = 5;
4     int result = a + b + constant;
5     return result; // apenas o valor de retorno é mantido em memória
6 }
7
8 int main(int argc, char** argv) {
9     int a = 2;
10    int b = 3;
11    const_sum_values(a,b);
12 }
```



```

1 // memória gerida "automaticamente" pelo programa
2 int* build_stack_array() {
3     int arr[10]; // espaço para 10 inteiros é alocado em stack; este espaço será dealocado no fim da
4     ↪ execução da função
5     int* arr_pointer = arr;
6     return arr_pointer;
7 }
8
9 // memória gerida "manualmente" pelo programador, na Heap
10 int* build_heap_array() {
11     int* num_pointer_heap; // um apontador não inicializado; neste momento aponta para um local
12     ↪ aleatório da memória
13     num_pointer_heap = malloc(10 * sizeof(int)); // a função malloc aloca um bloco de memória para
14     ↪ 10 elementos do tipo int em memória heap e retorna o endereço para o início desse bloco;
15     ↪ esse endereço é guardado no apontador anteriormente definido
16     return num_pointer_heap;
17 }
18
19 int main () {
20     int* num_stack;
21     int* num_heap;
22
23     num_stack = build_stack_array(); // aponta para uma zona de memória desalocada, não pode ser
24     ↪ usado
25     num_heap = build_heap_array(); // aponta para um array em zona de memória heap; o array pode ser
26     ↪ usado enquanto não for desalocado
27
28     free(num_heap); // dealoca o espaço anteriormente alocado pela função malloc
29 }

```

1.8 #define e typedef

O operador `#define` cria um alias (ou macro) para variáveis, tipos ou funções de um programa. Antes do programa ser compilado, o pré-processador substitui no texto do programa esses alias, funcionando no fundo como um mecanismo de "copy+paste". Por convenção, os alias atribuídos são escritos em letras maiúsculas.

```

1 #define ONE 1
2
3 int main(int argc, char** argv) {
4     // antes do programa ser compilado, o termo ONE vai ser substituído por 1
5     int a = ONE;
6     return 0;
7 }

```

O operador `typedef` permite renomear tipos existentes, criando um novo tipo.

```

1 typedef long int lu;
2
3 int main(int argc, char** argv) {
4     lu a = 1000;
5     printf("Long int has value: %ld\n",a);
6     return 0;
7 }

```

Este operador é bastante útil quando estamos a lidar com structs. Vejamos a definição abaixo:

```

1 struct item {

```

```

2   char name[50];
3   float basePrice;
4   float stock;
5   float discount;
6 };

```

De momento, precisamos de indicar `struct item` para representar o tipo de um item. Contudo, isto pode ser simplificado através do uso do `typedef`:

```

1 typedef struct item Item;

```

Também pode ser feito diretamente na definição da struct:

```

1 typedef struct item {
2     char name[20];
3     ...
4 } Item;

```

Com isto, o produto já pode ser declarado da seguinte forma:

```

1 Item i;

```

Quando estamos a lidar com tipos, não devemos usar `#define`. Este operador não define um novo tipo, apenas substitui texto, substituindo todas as instâncias de uma palavra, independentemente de se tratarem de tipos ou variáveis, podendo levar a erros complicados de detetar. O operador `typedef` garante que o novo tipo estará sujeito a regras de escopo e de sintaxe, e será interpretado pelo compilador como qualquer outro tipo. Vejamos os exemplos abaixo, que comparam os dois casos. A função `concatenate_bytes` concatena dois bytes guardados numa struct, e devolve um inteiro de valor correspondente a essa concatenação.

```

1 #define byte unsigned char
2
3 int main(int argc, char** argv) {
4     byte a;
5     int byte = 3;
6
7     a = byte;
8     return 0;
9 }

```

Após passar pelo pré-processador, a função main ficaria com o seguinte aspeto:

```

1 // código obviamente não compilável
2 int main(int argc, char** argv) {
3     unsigned char a;
4     int unsigned char = 0;
5     a = unsigned char;
6
7     return 0;
8 }

```

Já se usarmos antes `typedef unsigned char byte`; o programa funcionará da forma esperada.

1.9 Estruturas de dados abstratas

As estruturas de dados abstratas são estruturas definidas pelo seu comportamento, podendo ter diversas implementações. Uma das estruturas abstratas mais usadas é a `List`, que representa uma sequência de valores onde a ordem é relevante. Para representar estruturas de dados abstratas em C, usamos o `void*`. Por exemplo, podemos implementar uma simples `List` da seguinte forma:

```
1 typedef struct list {
2     void* data;
3     struct list* next;
4 } List;
```

Para adicionar qualquer valor à cabeça lista, podemos usar a seguinte função:

```
1 List* create(void* data) {
2     List* new = malloc(sizeof(List));
3     new->data = data;
4     new->next = NULL;
5     return new;
6 }
7
8 List* prepend(List* l, void* data) {
9     List* new = create(data);
10    new->next = l;
11    return new;
12 }
```

Funções por referência

Para efetuar operações sobre estruturas de dados abstratas são muitas vezes usadas funções de ordem superior, ou seja, funções que recebem outras funções por argumento. Considere a função `apply`, que aplica uma função a todos os elementos da nossa lista:

```
1 void apply(List* l, void (*f)(void*)) {
2     while (l) {
3         f(l->data);
4         l = l->next;
5     }
6 }
```

Neste caso, `apply` recebe uma função `f` que por sua vez recebe `void*` por argumento e não retorna nada, aplicando-a a todos os elementos da lista. Um exemplo de `f` é a função `square`, que converte um número no seu quadrado:

```
1 void square(void* i) {
2     int* i_ = i; // precisamos fazer cast para calcular o quadrado
3     *i_ *= *i_;
4 }
```

Considerando a lista `*l` com valores `[10, 20, 30, 40]`, aplicar `apply(l, &square)` irá resultar nos novos valores `[100, 400, 900, 1600]`.

Exemplos com diferentes tipos de dados

O campo `void* data` permite armazenar qualquer tipo de dados. Exemplos de uso:

```

1 // inteiros
2 int* num = malloc(sizeof(int));
3 *num = 42;
4 List* int_list = create(num);
5
6 // floats
7 float* f = malloc(sizeof(float));
8 *f = 3.14;
9 List* float_list = create(f);
10
11 // strings
12 char* str = malloc(20);
13 strcpy(str, "Hello");
14 List* string_list = create(str);
15
16 // structs
17 typedef struct { char name[30]; int age; } Person;
18 Person* p = malloc(sizeof(Person));
19 strcpy(p->name, "Alice");
20 p->age = 25;
21 List* person_list = create(p);
22
23 // listas aninhadas
24 List* nested_list = create(int_list);

```

É essencial fazer o cast adequado de `void*` para o tipo correto e gerir a memória apropriadamente. No limite, até poderíamos ter listas polimórficas, i.e., listas que contêm elementos de tipos diferentes.

1.10 Ficheiros .h vs .c

Declaração vs definição de funções

Há uma distinção importante entre declaração e definição de uma função. A **declaração** de uma função indica o tipo de valor retornado pela função, e o número e tipo de argumentos que essa função recebe, mas não inclui as operações efetuadas pela função. A **definição** de uma função, para além de incluir a sua declaração, define a lógica da função, ou seja, define a sequência de operações a serem executadas pela função.

```

1 // declaração de função
2 int sum(int a, int b);
3
4 // definição de função
5 int sum(int a, int b) {
6     return a+b;
7 }

```

Ficheiros .h

Os ficheiros **.h** contêm **declarações de funções e definições de tipos (entre outros) que queremos partilhar** com outros ficheiros. Funcionam como a definição de uma interface, que explicita a quem quiser usar o nosso código as funções que estamos a disponibilizar, e eventualmente novos tipos de variáveis que são retornadas pelas nossas funções. Assim, evitamos ter que definir uma função em todos os ficheiros que precisam dela, promovendo assim a reutilização de código, já que só temos que definir a função uma vez. Para além disso, permite-nos também restringir as funções que outros ficheiros podem usar.

Um ficheiro **.h** deve ser importado tanto por quem quer usar as funções que ele declara, como pelos ficheiros que implementam (i.e. definem) essas funções.

Para além disso, qualquer ficheiro **.h** deve começar pelas diretivas `#ifndef` e `#define`, seguidas de um nome único, de modo a evitar reimportar o ficheiro múltiplas vezes, o que levaria a erros de compilação. Aqui adotamos a convenção de usar

o nome do ficheiro, em maiúsculas e unido por `_` terminando em `_H`. O ficheiro deve ainda terminar em `#endif`. Veja-se o exemplo abaixo:

```

1  #ifndef FICHEIRO_A_H
2  #define FICHEIRO_A_H
3
4  void func1();
5  int func2(char a);
6  ...
7
8  #endif

```

Ficheiros .c

Os ficheiros `.c` devem conter as definições (i.e., implementações) de funções do programa. Devem também incluir todos os tipos e variáveis que não queremos que sejam partilhadas com outros ficheiros. Uma das vantagens desta estruturação é que podemos ter diferentes implementações (usando diferentes `.c`) para diferentes interfaces, podendo optar por uma ou por outra dependendo das nossas necessidades, na altura da compilação.

Por exemplo, considere novamente a lista genérica definida acima. A sua interface conterá a declaração da estrutura e das operações a implementar:

```

1  // ficheiro list.h
2  #ifndef LIST_H
3  #define LIST_H
4  typedef struct list {
5      void* data;
6      List* next;
7  } List;
8
9  List* create_list(void* data);
10 List* prepend_list(List* l, void* data);
11 ...
12 #endif

```

Já o ficheiro com a implementação irá primeiro importar a interface e de seguida implementar as operações descritas:

```

1  // ficheiro list.c
2  #include "list.h"
3
4  List* create_list(void* data) {
5      ...
6  }
7
8  List* prepend_list(List* l, void* data) {
9      ...
10 }
11
12 ...

```

Esta lista poderá depois ser utilizada noutros ficheiros simplesmente importando o ficheiro header:

```

1  // ficheiro main.c
2  #include "list.h"
3
4  int main() {
5      int* a = malloc(sizeof(int));

```

```

6   List* list = create_list(a);
7   return 0;
8 }
9
10 ...

```

1.11 Compilação e execução

Um programa passa por 4 ferramentas antes de se tornar num executável, como descrito na Figura 3, nomeadamente o Pré-processador, o Compilador, o Assembler e o Linker.

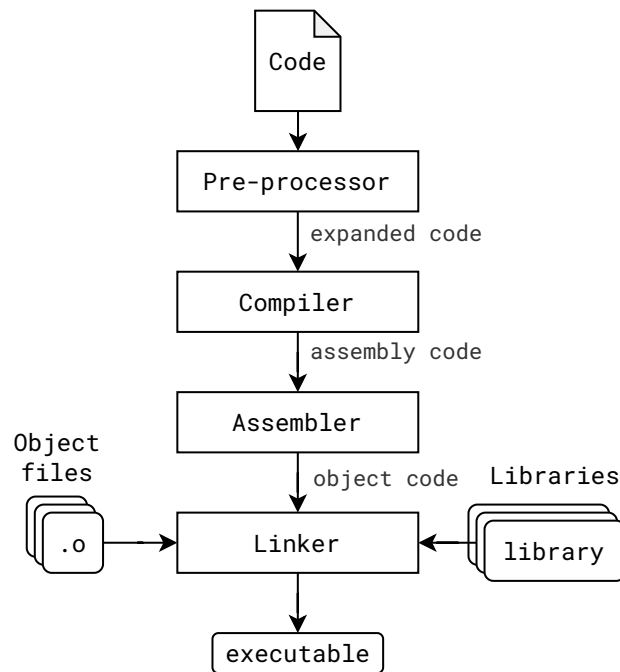


Figura 3: Passos do processo de compilação.

Adaptado de <https://www.javatpoint.com/compilation-process-in-c>

O pré-processador efetua múltiplas funções. Entre estas inclui-se a remoção de todos os comentários do ficheiro e a substituição de macros definidas com `#define` pelo texto correspondente. Para além disso, é feita a substituição de diretivas `#include` pelo conteúdo desses ficheiros, como indicação futura de que as funções declaradas nesse ficheiro deverão ser procuradas em ficheiros externos.

O compilador converte o output produzido pelo pré-processador para código Assembly.

O Assembler, por sua vez, converte o ficheiro de código Assembly num *object file*, que contém código máquina (ou binário).

Quando temos um projeto com múltiplos ficheiros, cada ficheiro `.c` irá originar um ficheiro de código máquina `.o`. E cada ficheiro `.c` poderá fazer uso de funções que não estão definidas no próprio ficheiro. Para combinar as definições das diferentes funções que se encontram nos diferentes ficheiros, de forma a formarmos um ficheiro executável com todo o código necessário, recorreremos ao Linker. O Linker combina os diferentes ficheiros `.o` do nosso projeto, bem como ficheiros `.o` de bibliotecas que estejam a ser usadas no nosso programa, para formar o executável final.

Considere o exemplo onde temos o ficheiro `list.h`, que define a interface de uma lista, e o ficheiro `list.c`, que define uma possível implementação. Considere ainda a existência do ficheiro `main.c`, que usa a lista declarada em `list.h`. O primeiro passo a tomar consiste na geração dos ficheiros `.o`, através do comando `gcc -c`:

```
$ gcc -c list.c # gera object file list.o
```

De seguida, geramos o executável `main`, fornecendo o ficheiro `list.o` como argumento:

```
$ gcc main.c list.o -o main
```

Considere agora uma segunda implementação de `list.h`, definida em `list_fast.c`, que consome mais memória mas obtém tempos de execução mais rápidos. Para usar esta nova implementação, simplesmente trocamos a implementação de `list.h` no momento da compilação:

```
$ gcc -c list_fast.c # gera object file list_fast.o
$ gcc main.c list_fast.o -o main
```

Não foi necessário modificar o ficheiro que usa `list.h`, visto que a declaração da interface não foi alterada.

2 Exemplo prático

O exemplo de seguida consiste na criação e teste de uma `Stack` genérica. A `Stack` é uma estrutura de dados que suporta a operações de `push` – adicionar um elemento no top – e `pop` – retira o elemento do top, retornando-o.

Começemos por definir as interfaces do nosso exemplo. A primeira refere-se a `Node`, que representa o elemento base da nossa `Stack`:

```

1 // ficheiro node.h
2 #ifndef NODE_H
3 #define NODE_H
4
5 typedef struct node {
6     void* data;
7     struct node* next;
8 } Node;
9
10 // cria um novo Node
11 Node* nodeCreate(void* data);
12
13 #endif

```

A segunda é a `Stack`, que contém os elementos e um contador, e declara diversas operações de modificação e leitura:

```

1 // ficheiro stack.h
2 #ifndef STACK_H
3 #define STACK_H
4
5 #include "node.h"
6
7 typedef struct stack {
8     int size;
9     Node* top;
10     void (*elemPrint)(void *); // função para imprimir um elemento da Stack
11 } Stack;
12
13 Stack* stackCreate(void (*elemPrint)(void *));
14 void stackPush(Stack* stack, void* data);
15 void* stackPop(Stack* stack);
16 void stackPrint(Stack* stack);
17
18 #endif

```

De seguida, passámos à implementação da duas interfaces:

```

1 // ficheiro node.c
2 #include "node.h"
3 #include <stdlib.h>
4
5 Node* nodeCreate(void* data) {
6     Node* new = malloc(sizeof(Node));
7     new->data = data;
8     new->next = NULL;
9     return new;
10 }

```

```

1 // ficheiro stack.c
2
3 #include "stack.h"

```



```

4  #include <stdio.h>
5  #include <stdlib.h>
6
7  Stack* stackCreate(void (*elemPrint)(void *)) {
8      Stack* stack = malloc(sizeof(Stack));
9      stack->size = 0;
10     stack->top = 0; // o mesmo que NULL
11     stack->elemPrint = elemPrint;
12     return stack;
13 }
14
15 void stackPush(Stack* stack, void* data) {
16     Node* node = nodeCreate(data);
17     node->next = stack->top;
18     stack->top = node;
19     stack->size++;
20 }
21
22 void* stackPop(Stack* stack) {
23     if (stack->size == 0) {
24         return 0; // o mesmo que NULL
25     }
26
27     Node* top = stack->top;
28     stack->top = top->next;
29     void* data = top->data;
30     free(top);
31     stack->size--;
32     return data;
33 }
34
35 void stackPrint(Stack* stack) {
36     Node* top = stack->top;
37     while (top) {
38         stack->elemPrint(top->data);
39         printf(" -> ");
40         top = top->next;
41     }
42     printf("x\n");
43 }

```

Por último, criamos uma `main` para testar a estrutura. Neste caso, começamos por criar a `Stack`, alocando e inserindo 5 inteiros aleatórios. Depois, removemos e libertamos os elementos um a um. Finalmente, libertamos a `Stack`:

```

1  // ficheiro main.c
2  #include "stack.h"
3  #include <time.h>
4  #include <stdlib.h>
5  #include "stdio.h"
6  #include "stack.h"
7
8  void printInt(void* i) {
9      int* i_ = i;
10     printf("%d", *i_);
11 }
12
13 int main(int argc, char** argv) {
14     srand(time(NULL)); // seed do random, para ser diferente a cada run
15
16     Stack* s = stackCreate(&printInt);
17     for (int i = 0; i < 5; i++) {

```

```

18     stackPrint(s);
19     int* i = malloc(sizeof(int));
20     *i = rand() % 100; // [0, 100[
21     stackPush(s, i);
22 }
23 stackPrint(s);
24
25 int* i;
26 // termina quando retornar NULL, ou seja,
27 // não existe mais elementos para remover
28 while (i = stackPop(s)) {
29     free(i); // é preciso libertar os dados pois stackPop liberta apenas o nó
30     stackPrint(s);
31 }
32
33 free(s);
34
35 return 0;
36 }

```

Para gerar um executável, precisamos primeiro de compilar os vários módulos usados pela nossa `main`. De seguida, compilamos o ficheiro com a `main`, indicando a implementação das dependências:

```

$ gcc -c node.c
$ gcc -c stack.c
$ gcc main.c node.o stack.o -o main

```

Executando `./main`, obtemos o seguinte output:

```

$ ./main
x
8 -> x
70 -> 8 -> x
41 -> 70 -> 8 -> x
35 -> 41 -> 70 -> 8 -> x
99 -> 35 -> 41 -> 70 -> 8 -> x
35 -> 41 -> 70 -> 8 -> x
41 -> 70 -> 8 -> x
70 -> 8 -> x
8 -> x
x

```

3 Exercícios

Considere a estrutura de dados abstrata `Deque` (*double-ended queue*), que modela uma fila onde valores podem ser inseridos no início ou no fim, podendo ainda ser removidos do início ou do fim (Figura 4).

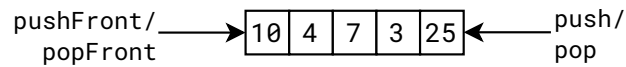


Figura 4: Exemplo de uma `Deque`.

A `Deque` deverá suportar as seguintes operações:

- `Deque* create()` – inicializa uma `Deque` sem elementos;
- `void push(D deque, void* data)` – adiciona `data` no fim da fila;
- `void pushFront(D deque, void* data)` – adiciona `data` no início da fila;
- `void* pop(D deque)` – remove o elemento do fim da fila e retorna-o; retorna `NULL` se a fila não tiver elementos;
- `void* popFront(D deque)` – remove o elemento do início da fila e retorna-o; retorna `NULL` se a fila não tiver elementos;
- `int size(D deque)` – retorna o número de elementos da fila;
- `bool isEmpty(D deque)` – indica se a fila está vazia;³
- `void reverse(D deque)` – inverte a ordem dos elementos da fila;
- `void printDeque(D deque, void(*printFunc)(void*))` – imprime os elementos da fila com o auxílio da função `printFunc`;
- `void destroy(D deque)` – liberta a memória associada à `Deque`.

O design da `Deque` deverá permitir que as operações acima sejam executadas com o número mínimo de iterações possíveis.

1. Defina a interface da `Deque`;
2. Implemente a `Deque`;
3. Crie uma função para testar a estrutura, inserindo/removendo valores (e.g., `int*`) no início ou no fim da `Deque` de forma aleatória, imprimindo para a consola o seu estado a cada operação.

³Para usar o tipo de dados `bool` é necessário incluir `stdbool.h`

4 Conceitos adicionais

Enums

Os enums são enumerações de valores constantes inteiros. São usados para representar categorias, estados, tipos, etc., de forma simples. Por exemplo, o seguinte enum representa o gênero de um filme:

```
1 enum genre {
2     Action,
3     Comedy,
4     Drama,
5     ...
6 }
```

Os valores de um enum podem ser implícitos, como exemplificado em cima, ou explícitos, com mostra o exemplo seguinte:

```
1 enum genre {
2     Action = 0,
3     Comedy = 1,
4     Drama = 2,
5     ...
6 }
```

De seguida, apresenta-se o uso do `enum genre` numa struct de filmes:

```
1 struct movie {
2     char name[50];
3     int runtime;
4     enum genre genre;
5 }
6
7 struct movie m;
8 m.genre = Comedy;
```

union

O construtor `union` permite definir variáveis que partilham o mesmo espaço de memória. Ao definir uma variável `union`, será alocado espaço suficiente para manter a maior das variáveis dessa `union`. Ao usar uma `union`, o único valor válido será o da última variável escrita, uma vez que escrever uma das variáveis irá corromper a representação das restantes. Na prática, uma variável do tipo `union` pode ser vista como uma única variável que pode adotar diferentes tipos.

Uma alternativa ao uso de variáveis `union` seria fazer uso de `void *`, o que inclusive evitaria ter uma variável que ocupa mais espaço do que o necessário. No entanto, com o uso de `union` garantimos que a variável apenas poderá adotar um dos tipos definidos na `union`. Torna também possível rapidamente alternar o tipo da variável a ser representada, sem ser necessário voltar a alocar memória:

```
1 // o tamanho de uma variável deste tipo será sempre 8 bytes (o maior na union)
2 union classification {
3     int i;
4     float f;
5     char s[8];
6 }
7
8 int main() {
9     union classification clf;
10    clf.i = 10;
11    clf.f = 0.1;
12    strcpy(clf.s, "Bad");
```

```

13 // apenas o valor de clf.s é válido neste ponto do código
14 return 0;
15 }

```

Modificadores de armazenamento

O *scope* e tempo de vida de variáveis pode ser controlado através do uso de modificadores de armazenamento (Tabela 1).

Modificador	Scope	Tempo de vida
auto	Bloco	Execução da função
static	Ficheiro	Fim do programa
register	Bloco	Execução da função
extern	Programa	Fim do programa

Tabela 1: Modificadores de armazenamento

De um modo geral os modificadores `extern` e `register` **não deverão ser usados**. O modificador `extern`, em particular, introduz a possibilidade de declaração e uso incorreto de funções externas, situação de erro particularmente difícil de identificar e depurar. O modificador `auto` é o modificador aplicado por defeito a variáveis **locais**, e não deve ser aplicado a variáveis globais, logo para os propósitos desta UC pode ser ignorado.

O modificador `static` permite-nos estender o tempo de vida de uma variável local até ao fim do programa. Quando usado em variáveis locais, a variável é inicializada apenas na primeira chamada da função e o valor da variável será mantido entre chamadas.

```

1 void func1() {
2     static int a = 0;
3     a = a + 2;
4 }
5
6 int main(int argc, char** argv) {
7     func1();
8     func1();
9     // variável a da func1 tem agora o valor 4
10 }

```